

- The concept of PAC learnability is interesting.
- It allows one to properly define what is correctness of learning and allows us to ask questions like whether a given algorithm learns correctly.
- As we have seen, we can also bound the number of examples needed to learn to a given level of accuracy and confidence.
Called sample complexity of a learning problem.
- Thus we can appreciate relative complexities of different learning problems.



In our new framework we are given

- $\mathcal{X}$ – input space; (as earlier, *Feature space*)
- $\mathcal{Y}$ – Output space (as earlier, *Set of class labels*)
- $\mathcal{H}$ – hypothesis space (*family of classifiers*)

Each $h \in \mathcal{H}$ is a function: $h : \mathcal{X} \rightarrow \mathcal{A}$
where $\mathcal{A}$ is called *action space*.

- Training data: $\{(X_i, y_i), i = 1, \dots, n\}$
drawn *iid* according to some distribution P_{xy} on $\mathcal{X} \times \mathcal{Y}$.



Some Comments

- We have replaced $\mathcal{C}$ with $\mathcal{H}$.
- If we take $\mathcal{A} = \mathcal{Y}$ then it is same as earlier.
- But the freedom in choosing $\mathcal{A}$ allows for taking care of many situations.
- For example, even when $\mathcal{Y} = \{0, 1\}$, we can take $\mathcal{A} = \mathbb{R}$ (e.g., learning discriminant functions).
- Now, e.g., sign of $h(X)$ may denote the class and its magnitude may give some measure of confidence in the assigned class.



- Now we draw examples from $\mathcal{X} \times \mathcal{Y}$ according to P_{xy} . This allows for 'noise' in the training data.
- For example, when class conditional densities overlap, same X can come from different classes with different probabilities.
- We can always factorize $P_{xy} = P_x P_{y|x}$. If $P_{y|x}$ is a degenerate distribution then it will be same as earlier – we draw iid samples from $\mathcal{X}$ and each point is essentially classified by the target classifier.
- However, having examples drawn from $\mathcal{X} \times \mathcal{Y}$ using a distribution, allows for many more scenarios.



- As before, the learning machine outputs a hypothesis, $h_n \in \mathcal{H}$, given the training data consisting of n examples.
- However, now there is no notion of a target concept/hypothesis.
- There may be no $h \in \mathcal{H}$ which is consistent with all examples.
- Hence we use the idea of loss functions to define the goal of learning.



Loss function

- Loss function: $L : \mathcal{Y} \times \mathcal{A} \rightarrow \mathbb{R}^+$.
- The idea is that $L(y, h(X))$ is the 'loss' suffered by $h \in \mathcal{H}$ on a (random) sample $(X, y) \in \mathcal{X} \times \mathcal{Y}$.
- More generally we can let loss depend on X also explicitly and can write $L(X, y, h(X))$ for loss function.
- By convention we assume that the loss function is non-negative.
- Now we can look for hypotheses that have low average loss over samples drawn according to P_{xy} .



Risk function

- Define a function, $R : \mathcal{H} \rightarrow \mathbb{R}^+$ by

$$R(h) = E[L(y, h(X))] = \int L(y, h(X)) dP_{xy}$$

- R is called the risk function.
- Risk is expectation of loss where expectation is with respect to P_{xy} .
- Hence, h with a low $R(h)$ is a better classifier.
- We can now define the goal of learning as minimizer of risk.



- Let $h^* = \arg \min_{h \in \mathcal{H}} R(h)$
- We define the goal of learning as finding h^* , the global minimizer of risk.
- Risk minimization is a very general strategy adopted by most machine learning algorithms.
- However, note that we may not have any knowledge of P_{xy} .
- How can we find minimizer of risk? Minimization of $R(\cdot)$ directly is not feasible.



Empirical Risk function

- Define the empirical risk function, $\hat{R}_n : \mathcal{H} \rightarrow \mathbb{R}^+$, by

$$\hat{R}_n(h) = \frac{1}{n} \sum_{i=1}^n L(y_i, h(X_i))$$

This is the sample mean estimator of risk obtained from n iid samples.

- Let $\hat{h}_n^*$ be the global minimizer of empirical risk, $\hat{R}_n$.

$$\hat{h}_n^* = \arg \min_{h \in \mathcal{H}} \hat{R}_n(h)$$



- The loss function is known to us.
- Hence, given any h we can calculate $\hat{R}_n(h)$.
- Hence, we can (in principle) find $\hat{h}_n^*$ by optimization methods.
- Approximating h^* by $\hat{h}_n^*$ is the basic idea of empirical risk minimization strategy which is used in most ML algorithms.



- Is $\hat{h}_n^*$ a good approximator of h^* , the minimizer of true risk (for large n)?
- This is the question of *consistency* of empirical risk minimization.
- Thus, we can say that any learning problems has two parts.
- The optimization part: find $\hat{h}_n^*$, the minimizer of $\hat{R}_n$.
The statistical part: Is $\hat{h}_n^*$ a good approximator of h^* .



- We will now look at some examples of loss functions.
- Note that the loss function is chosen by us; it is part of the specification of the learning problem.
- The loss function is intended to capture how we would like to evaluate performance of the classifier.



- The 0-1 loss function is

$$L(y, h(X)) = I_{[y \neq h(X)]}$$

- Risk is expectation of loss.
- Hence, $R(h) = \text{Prob}[y \neq h(X)]$; the risk is probability of misclassification.
- So, h^* minimizes probability of misclassification. A very desirable goal. (e.g., Bayes classification)



- Here we assumed that the learning algorithm searches over a class of binary-valued functions on $\mathcal{X}$.
- We can extend this to, e.g., discriminant function learning.
- We take $\mathcal{A} = \mathfrak{R}$ (now $h(X)$ is a discriminant function).
- We can define the 0-1 loss now as

$$L(y, h(X)) = I_{[y \neq \text{sgn}(h(X))]}$$



- Having any fixed misclassification costs is essentially same as 0–1 loss.
- Even if we take $\mathcal{A} = \mathbb{R}$, the 0–1 loss compares only sign of $h(x)$ with y . The magnitude of $h(x)$ has no effect on the loss.
- Here, we can not trade ‘good’ performance on some data with ‘bad’ performance on others.
- This makes 0–1 loss function more robust to noise in classification labels.



- While 0–1 loss is an intuitively appealing performance measure, minimizing empirical risk here is hard.
- Note that the 0–1 loss function is non-differentiable which makes the empirical risk function also non-differentiable.
- The empirical risk minimization is also a non-convex optimization problem here.
- Hence many other loss functions are often used in Machine Learning.



Squared error loss

- The squared error loss function is defined by

$$L(y, h(X)) = (y - h(X))^2$$

- As is easy to see, the linear least squares method that we considered is empirical risk minimization with squared error loss function.
- Here, for a 2-class classification problem, we can take $\mathcal{Y}$ as $\{0, 1\}$ or $\{+1, -1\}$. We take $\mathcal{A} = \mathbb{R}$ so that each h is a discriminant function.
- As we know, we can use this for regression problems also and then we take $\mathcal{Y} = \mathbb{R}$.

- Another interesting scenario here is to take $\mathcal{Y} = \{0, 1\}$ and $\mathcal{A} = [0, 1]$.
- Then each h can be interpreted as a posterior probability (of class-1) function.
- As we know, the minimizer of expectation of squared error loss (the risk here) is the posterior probability function.
- So, risk minimization would now look for a function in $\mathcal{H}$ that is a good approximation for the posterior probability function.



soft margin loss or hinge loss

- We take $\mathcal{Y} = \{+1, -1\}$ and $\mathcal{A} = \mathbb{R}$. The loss function is given by

$$L(y, h(X)) = \max(0, 1 - yh(X))$$

- Here, if $yh(X) > 0$ then classification is correct and if $yh(X) \geq 1$, loss is zero.
- This also results in convex optimization for empirical risk minimization.



- We can look at the other loss functions as some convex approximations of the 0–1 loss functions as follows.
- Let us take $\mathcal{Y} = \{+1, -1\}$.
- We can write all loss functions as functions of single variable $yh(X)$.



- For 0–1 loss $L(y, h(X))$ is one if $yh(X)$ is negative and zero otherwise.
- The squared error loss can be written as

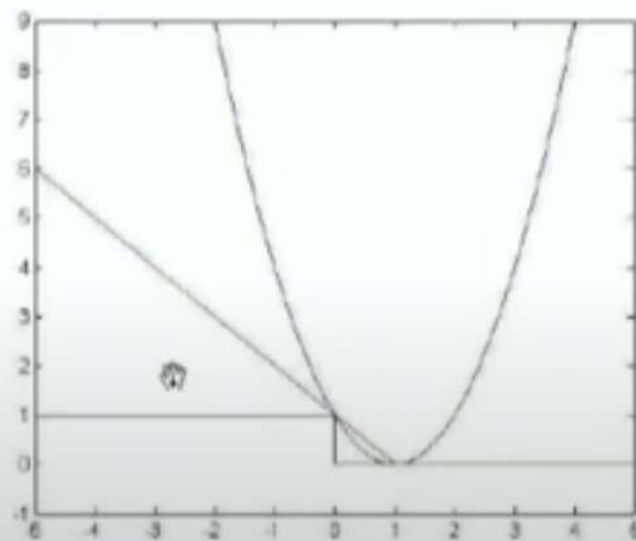
$$L(y, h(X)) = (1 - yh(X))^2$$

- The hinge loss is defined as a function of $yh(X)$.

$$L(y, h(X)) = \max(0, 1 - yh(X))$$



- We can plot all the functions as follows.



(Here we plot $yh(X)$ on x -axis and $L(y, h(X))$ on y -axis).

- There are also many other loss functions used in regression problems.
- The squared error loss can be used. But it is very sensitive to data errors such as outliers.
- So, other loss functions are used for better robustness during regression.



- The L_1 loss is defined by

$$L(y, h(X)) = |y - h(X)|$$

- This is more robust than the square loss.
- Another similar one is the ϵ -insensitive loss defined by

$$L(y, h(X)) = \max(0, |y - h(X)| - \epsilon)$$

- Here if we make error less than epsilon then loss is zero.



- As we saw, there are many different loss functions one can think of.
- Many of them also make the empirical risk minimization problem efficiently solvable.
- Thus a choice of loss function can efficiently tackle the optimization part of learning.
- We consider some such algorithms later in this course.
- Now, let us get back to the ^⑤statistical question that we started with.



Consistency of Empirical Risk Minimization

- Our objective is to find h^* , minimizer of risk $R(\cdot)$.
- Since we do not know R , we minimize the empirical risk, $\hat{R}_n$, instead and thus find $\hat{h}_n^*$.
- We want h^* and $\hat{h}_n^*$ to be 'close'. More precisely we are interested in the question

$$R(\hat{h}_n^*) \rightarrow R(h^*)?$$

(As earlier, we know $\hat{h}_n^*$ is random and hence we take the above as convergence in probability).



- What is the intuitive reason for using empirical risk minimization?
- Sample mean is a good estimator and hence, with large n , $\hat{R}_n(h)$ converges to $R(h)$, for any $h \in \mathcal{H}$.
- This is (weak) law of large numbers.
- But this does not necessarily mean $R(\hat{h}_n^*)$ converges to $R(h^*)$.
- We are interested in: does the true risk of the minimizer of empirical risk converge to global minimum of risk?



- Let us consider a specific scenario to appreciate this.
- We take $\mathcal{A} = \mathcal{Y} = \{0, 1\}$. We use 0–1 loss.
- Suppose the examples are drawn according to P_x on $\mathcal{X}$ and classified according to a $\tilde{h} \in \mathcal{H}$.
- That is $P_{xy} = P_x P_{y|x}$ and $P_{y|x}$ is a degenerate distribution.
- Now the global minimum of risk is zero and $R(\tilde{h}) = 0$.



- Note that now the risk of any h is same as $P_x(h\Delta\tilde{h})$.
- That is, this scenario is same as what we considered under PAC framework.
- Now, under 0–1 loss, the global minimum of empirical risk is also zero.
- For any n , there may be many h (other than $\tilde{h}$) with $\hat{R}_n(h) = 0$.
- Hence our optimization algorithm can only use some general rule to output one such hypothesis.



- Suppose we took $\mathcal{H} = 2^{\mathcal{X}}$.
- This is same as the example we considered earlier and hence we know, $P_x(\hat{h}_n^* \Delta \tilde{h})$ will not go to zero with n .
- Thus, here, $R(\hat{h}_n^*)$ will not converge to $R(h^*)$.
- Note that the law of large numbers still implies that $\hat{R}_n(h)$ converges to $R(h)$, $\forall h$.



- This shows that even though $\hat{R}_n(h) \rightarrow R(h)$, $\forall h$, the risk of minimizer of empirical risk need not converge to the global minimum of risk.
- And this happens based on our choice of $\mathcal{H}$.
- Hence, the statistical question we need to ask is: for what $\mathcal{H}$ is empirical risk minimization consistent.
- That is, given any $\epsilon, \delta > 0$, $\exists N < \infty$ such that

$$\text{Prob}[|R(\hat{h}_n^*) - R(h^*)| > \epsilon] \leq \delta ?$$

